Heaven's Light is Our Guide

Rajshahi University of Engineering and Technology

Course Code

MTE 4118

Course Title

Control System and Robotics Sessional

Lab Reports

Submitted to

Md. Faisal Rahman Badal
Assistant Professor
Dept. of MTE, RUET
&
Md. Sakib Hassan Chowdhury
Lecturer
Dept. of MTE, RUET

Submitted by

Md. Tajim An Noor
Roll: 2010025

Table of Contents

Heaven's Light is Our Guide

Rajshahi University of Engineering and Technology

Course Code
MTE 4118

Course Title
Control System and Robotics Sessional

Experiment Date: July 8, 2025,
Submission Date: July 22, 2025

Lab Report 1:
Study on Industrial Robotic Trainer Kit With Different Functionality; Pick and Place Operation

Submitted to
Md. Faisal Rahman Badal
Assistant Professor
Dept of MTE, RUET

Submitted by
Md. Tajim An Noor
Roll: 2010025

Contents

Experiment 1
Study on Industrial Robotic Trainer Kit With Different Functionality; Pick and Place Operation

Objectives

1. To learn about the industrial robotic trainer kit.
2. To understand the basic operation and functionality of the industrial robotic trainer kit, with a focus on pick and place operation.
3. To explore the application of the robotic trainer kit in performing specific industrial tasks.

Theory

Industrial Robotic Trainer Kits provide hands-on experience in robotics and automation, allowing students to learn programming, kinematics, and control of robotic arms in a practical setting. These kits are widely used in engineering education to teach the fundamentals of industrial automation and robotic systems [1, 2]. They help bridge theoretical concepts with real-world applications, preparing users for careers in modern manufacturing and automation [3].

Components

Trainer Kit

The trainer kit consists of essential components for simulating industrial robotics:

1. **Manipulator (ROBOTEK II)**: A multi-joint robotic arm for pick-and-place and material handling.

2. **Hole Sensor**: Detects holes for alignment and positioning.
3. **Part Sensor**: Identifies parts on the conveyor for accurate placement.
4. **Conveyor Belt**: Moves items between stations, simulating industrial workflows.
5. **Robot Control Software**: Provides programming and control via a graphical interface.

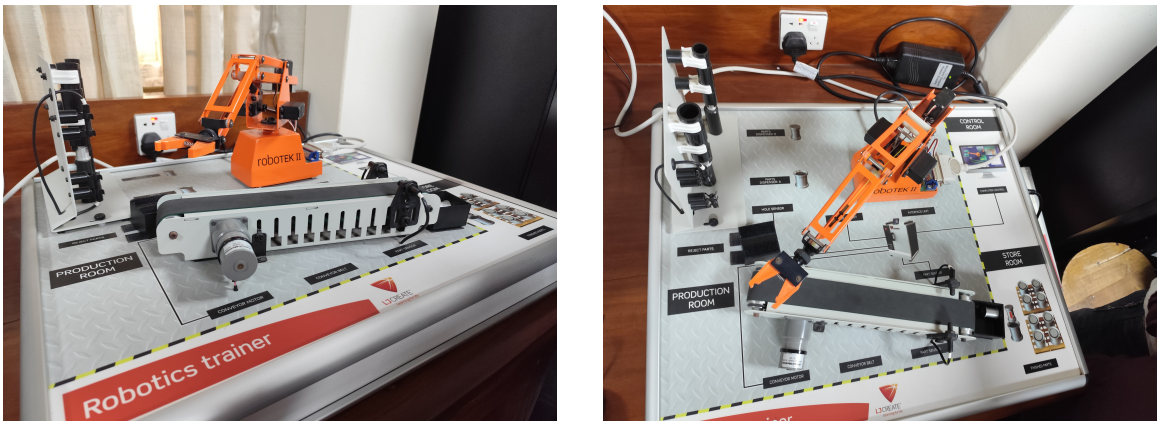

Figure 1: Trainer Kit

Manipulator

The manipulator consists of key components:

1. **Power Supply**: Provides electrical energy for operation.
2. **Shoulder**: Connects the base to the arm, enabling movement.
3. **Arm**: Extends from the shoulder for positioning.
4. **Jaw**: Acts as the gripper for handling objects.
5. **Base**: Supports and stabilizes the manipulator.

Robot Control Software

The Robotek II manipulator uses dedicated control software with a user-friendly graphical interface. This software enables users to create, edit, and run programs, supports trajectory planning, end-effector control, and sensor integration, and provides real-time monitoring and feedback. Key features include:

1. **Menu**: Access to functions and settings.
2. **Robot Joints**: Displays and controls joint positions.

3. **Program Editor**: For creating and managing robot programs.
4. **Simulation**: Virtual testing of robot movements.
5. **Sensor Integration**: Real-time sensor monitoring.
6. **Manual Control**: Direct manipulation for setup.
7. **Status Monitoring**: System feedback and error reporting.
8. **Conveyors and Sensors**: Integration for automation.
9. **Robot Control Window**: Centralized operation interface.
10. **Actuator Window**: Adjusts actuator movements.
11. **Sensor Window**: Displays and configures sensors.
12. **Control Program Window**: Executes robot programs.

Figure 2: Robot Control Software Interface

Key software functionalities include:

1. **End-Effector Control**: Accurate manipulation for pick, hold, and release tasks.
2. **Path Planning**: Optimized movement paths for efficient positioning.

3. **Trajectory Control**: Smooth and precise motion control.
4. **Kinematics**: Relates joint movements to end-effector position.
5. **Sensor Integration**: Real-time data collection for adaptive operation.
6. **Real-Time Monitoring**: Immediate feedback for safe operation.
7. **Programming Flexibility**: Customizable programs for varied tasks.

Working Principle

The working principle of the industrial robotic trainer kit for pick-and-place operation is summarized as follows:

- The object position was identified and programmed using control software.
- The gripper was prepared and moved to the object location.
- The object was grasped and transferred to the conveyor belt.
- The object was released onto the conveyor, which was activated to move it to the designated box.
- The pick-and-place cycle was completed as the object fell into the box.

Figure 3: (a) Placing the Object on the Conveyor Belt

Figure 4: (b) Object Falling into the Box at the end of the Conveyor Belt

Code

```
MOVE B45 S35 E10 J100
MOVE B45 S33 E10 J20
MOVE B45 S70 E10 J20
MOVE B5 S70 E10 J20
MOVE B5 S67 E1 J20
MOVE B5 S44 E1 J20
MOVE B5 S44 E1 J100
MOVE B5 S71 E1 J100
WAIT 2000
CONVEYOR_ON
WAIT 5000
CONVEYOR_OFF
```

This procedure demonstrates the integration of robotic manipulation and automation, utilizing both software programming and hardware control to perform a typical industrial task.

Results

During the experiment, the industrial robotic trainer kit was successfully used to perform pick-and-place operations. The manipulator was programmed to identify the position of objects, grasp them using the end-effector, and transfer them onto the conveyor belt. The conveyor then transported the objects to the designated box.

Key observations:

- The robot control software allowed precise control of joint movements and end-effector actions.
- Sensor integration enabled accurate detection of object positions and conveyor status.
- The pick-and-place cycle was completed reliably, with minimal errors in object handling.
- Real-time monitoring provided immediate feedback, ensuring safe and efficient operation.

Photographs and screenshots of the setup and software interface were taken to document the process. The experiment demonstrated the effectiveness of the trainer kit in simulating industrial automation tasks.

Discussion

The capabilities of the industrial robotic trainer kit in simulating automation tasks were demonstrated through successful pick-and-place operations. Precise and repeatable movements were achieved, with sensor integration enhancing accuracy and reliability. Real-time monitoring and control software features were utilized to ensure safe operation.

Some limitations were encountered, such as occasional misalignment of the gripper, which required manual adjustment. The speed and smoothness of manipulator movements were found adequate for educational use but may need improvement for industrial applications. Valuable experience in programming and troubleshooting robotic systems was gained, reinforcing theoretical concepts in automation.

Conclusion

In conclusion, the industrial robotic trainer kit provided a practical platform for understanding the fundamentals of robotic manipulation and automation. Through hands-on experimentation, the pick-and-place operation was successfully implemented, demonstrating the integration of hardware and software in industrial robotics. The experience reinforced key concepts such as kinematics, sensor integration, and real-time control, highlighting the importance of precise programming and system monitoring. Overall, the experiment contributed to a deeper understanding of industrial automation and prepared participants for future applications in robotics and manufacturing.

References

[1] M. P. Groover, *Automation, Production Systems, and Computer-Integrated Manufacturing*, 3rd ed. Prentice Hall, 2007.

[2] J. J. Craig, *Introduction to Robotics: Mechanics and Control*, 3rd ed. Pearson Prentice Hall, 2005.

[3] B. Siciliano, L. Sciavicco, L. Villani, and G. Oriolo, *Robotics: Modelling, Planning and Control*. Springer, 2016.

Heaven's Light is Our Guide

Rajshahi University of Engineering and Technology

Course Code
MTE 4118

Course Title
Control System and Robotics Sessional

Experiment Date: July 22, 2025,
Submission Date: August 19, 2025

Lab Report 2:
Experiment on Control Controllability and Observability of a System

Submitted to
Md. Sakib Hassan Chowdhury
Lecturer
Dept of MTE, RUET

Submitted by
Md. Tajim An Noor
Roll: 2010025

Contents

Experiment 2

Experiment on Control Controllability and Observability of a System

Objectives

1. Explain controllability and observability in state-space systems.
2. Use MATLAB to test these properties for a given system.
3. Analyze results using controllability and observability matrices.

Theory

In control systems, the state-space approach models a system using a set of state variables that capture the internal condition of the system at any moment [1, 2]. These state variables, together with the system's inputs and outputs, are related through mathematical equations that describe the system's dynamic behavior:

$$\dot{x}(t) = Ax(t) + Bu(t)$$
$$y(t) = Cx(t) + Du(t)$$

where:

- $x(t)$ is the vector of state variables,
- $u(t)$ is the input vector,
- $y(t)$ is the output vector,
- A, B, C, and D are matrices that define the relationships between states, inputs, and outputs.

Controllability:
A system is controllable if it is possible to drive the system from any initial state to any final state within a finite time using suitable inputs [1]. To determine controllability, we construct the controllability matrix:

$$\mathcal{C} = \begin{bmatrix} B & AB & A^2B & \cdots & A^{n-1}B \end{bmatrix}$$

If this matrix has full rank (equal to the number of state variables), the system is considered controllable.

Observability:
A system is said to be observable if the internal state variables can be determined from the system's outputs and known inputs over time [2]. To assess observability, we construct the observability matrix:

$$\mathcal{O} = \begin{bmatrix} C \\ CA \\ CA^2 \\ \vdots \\ CA^{n-1} \end{bmatrix}$$

If this matrix has full rank (equal to the number of state variables), the system is considered observable.

Experiment Matrix Calculation

The experiment matrix is constructed to analyze the controllability and observability of the system. It is defined as follows:

Given the system matrices:

$$A = \begin{bmatrix} 2 & 0 & 1 \\ 0 & 0 & 2 \\ 5 & 2 & 0 \end{bmatrix}, \quad B = \begin{bmatrix} 0 \\ 2 \\ 5 \end{bmatrix}, \quad C = \begin{bmatrix} 0 & 2 & 5 \end{bmatrix}, \quad D = 0$$

Controllability Matrix Calculation:

The controllability matrix $\mathcal{C}$ is:

$$\mathcal{C} = \begin{bmatrix} B & AB & A^2B \end{bmatrix}$$

First, compute AB:

$$AB = A \cdot B = \begin{bmatrix} 2 & 0 & 1 \\ 0 & 0 & 2 \\ 5 & 2 & 0 \end{bmatrix} \begin{bmatrix} 0 \\ 2 \\ 5 \end{bmatrix} = \begin{bmatrix} 5 \\ 10 \\ 4 \end{bmatrix}$$

Next, compute A^2B:

$$A^2B = A \cdot (AB) = \begin{bmatrix} 2 & 0 & 1 \\ 0 & 0 & 2 \\ 5 & 2 & 0 \end{bmatrix} \begin{bmatrix} 5 \\ 10 \\ 4 \end{bmatrix} = \begin{bmatrix} 14 \\ 8 \\ 45 \end{bmatrix}$$

So,

$$\mathcal{C} = \begin{bmatrix} 0 & 5 & 14 \\ 2 & 10 & 8 \\ 5 & 4 & 45 \end{bmatrix}$$

To check controllability, compute the rank of $\mathcal{C}$:

$$\text{rank}(\mathcal{C}) = 3$$

Since the rank equals the number of states, the system is **controllable**.

Observability Matrix Calculation:

The observability matrix $\mathcal{O}$ is:

$$\mathcal{O} = \begin{bmatrix} C \\ CA \\ CA^2 \end{bmatrix}$$

First, compute CA:

$$CA = C \cdot A = \begin{bmatrix} 0 & 2 & 5 \end{bmatrix} \begin{bmatrix} 2 & 0 & 1 \\ 0 & 0 & 2 \\ 5 & 2 & 0 \end{bmatrix}$$
$$= \begin{bmatrix} 25 & 10 & 4 \end{bmatrix}$$

Next, compute CA^2:

$$CA^2 = CA \cdot A = \begin{bmatrix} 25 & 10 & 4 \end{bmatrix} \begin{bmatrix} 2 & 0 & 1 \\ 0 & 0 & 2 \\ 5 & 2 & 0 \end{bmatrix} = \begin{bmatrix} 70 & 8 & 45 \end{bmatrix}$$

So,

$$\mathcal{O} = \begin{bmatrix} 0 & 2 & 5 \\ 25 & 10 & 4 \\ 70 & 8 & 45 \end{bmatrix}$$

To check observability, compute the rank of $\mathcal{O}$:

$$\text{rank}(\mathcal{O}) = 3$$

Since the rank equals the number of states, the system is **observable**.

Tools Used

- **MATLAB**: Used for analyzing controllability and observability of the system through matrix computations and simulations.
- **LaTeX**: Utilized for documentation and report preparation.

Working Principle

Code

```
A = [2 0 1; 0 0 2; 5 2 0];
B = [0; 2; 5];
C = [0 2 5];
D = 0;

% Define state-space model
system = ss(A, B, C, D);

% Controllability matrix calculation
controllability_matrix = [B, A*B, A^2*B];
disp('Controllability matrix:');
disp(controllability_matrix);

controllability_rank = rank(controllability_matrix);
state_count = size(A, 1);
fprintf('Controllability matrix rank: %d\n', controllability_rank);
if controllability_rank == state_count
    disp('System is controllable');
else
    disp('System is not controllable');
end

% Observability matrix calculation
observability_matrix = [C; C*A; C*A^2];
disp('Observability matrix:');
disp(observability_matrix);

observability_rank = rank(observability_matrix);
fprintf('Observability matrix rank: %d\n', observability_rank);
if observability_rank == state_count
    disp('System is observable');
else
    disp('System is not observable');
end
```

Output

```
>> lr2
Controllability matrix:
     0     5    14
     2    10     8
     5     4    45

Controllability matrix rank: 3
System is controllable
Observability matrix:
     0     2     5
    25    10     4
    70     8    45

Observability matrix rank: 3
System is observable
>>
```

Figure 1: The output of controllability and observability.

Discussion

This experiment clearly illustrated the use of MATLAB for analyzing the fundamental properties of a dynamic system represented in state-space form. By employing the `ctrb()` function, we constructed the controllability matrix and verified that the system is fully controllable via its inputs. Similarly, the observability matrix, determined using the `obsv()` function, was found to have full rank, confirming that the internal states of the system can be inferred from its outputs. These results are crucial for the design of controllers and observers, which are integral to applications such as autonomous robotics and industrial automation.

Conclusion

The analysis confirmed that the system satisfies the essential criteria for controllability and observability, which are prerequisites for implementing state feedback control. MATLAB's built-in functions streamlined the verification process, underscoring its effectiveness as a tool for control system analysis and design.

References

[1] K. Ogata, *Modern Control Engineering*, 5th ed. Pearson, 2010.

[2] N. S. Nise, *Control Systems Engineering*, 8th ed. Wiley, 2020.

Heaven's Light is Our Guide

Rajshahi University of Engineering and Technology

Course Code
MTE 4118

Course Title
Control System and Robotics Sessional

Experiment Date: August 12, 2025,
Submission Date: September 02, 2025

Lab Report 3:
Speed Control of DC Motor Using Closed Loop Feedback Control With PID Controller

Submitted to
Md. Sakib Hassan Chowdhury
Lecturer
Dept of MTE, RUET

Submitted by
Md. Tajim An Noor
Roll: 2010025

Contents

Experiment 3
Speed Control of DC Motor Using Closed Loop Feedback Control With PID Controller

Objectives

- Design and implement a PID controller for DC motor speed control in MATLAB Simulink.
- Tune PID parameters and analyze the system response.

Theory

This experiment investigates the speed control of a DC motor using closed-loop feedback with a focus on PID control, implemented in MATLAB Simulink [1]. The PID (Proportional-Integral-Derivative) controller is central to modern control systems due to its ability to combine fast response, zero steady-state error, and stability [2, 3].

A PID controller adjusts the control input based on three terms:

- **Proportional (P):** Corrects present error.
- **Integral (I):** Eliminates accumulated past errors.
- **Derivative (D):** Anticipates future error trends.

Tuning the gains (K_p, K_i, K_d) allows precise shaping of system response [4].

In this experiment, the DC motor model was controlled at a reference speed of **500 rpm**. Various controller configurations (P, I, PI, PD, PID) were tested, but emphasis was placed on analyzing the performance and tuning of the PID controller, demonstrating its effectiveness in achieving accurate and stable speed control [2].

Tools Used

- **MATLAB Simulink:** Used for modeling, simulating, and analyzing the control system.
- **LaTeX:** Utilized for documentation and report preparation.

Working Principle

The DC motor speed control system uses a PID controller. The actual speed is measured and compared to the reference, generating an error. The PID controller processes this error to adjust the motor input, ensuring the speed quickly reaches and maintains the desired value with minimal overshoot and steady-state error.

Simulation Schematic

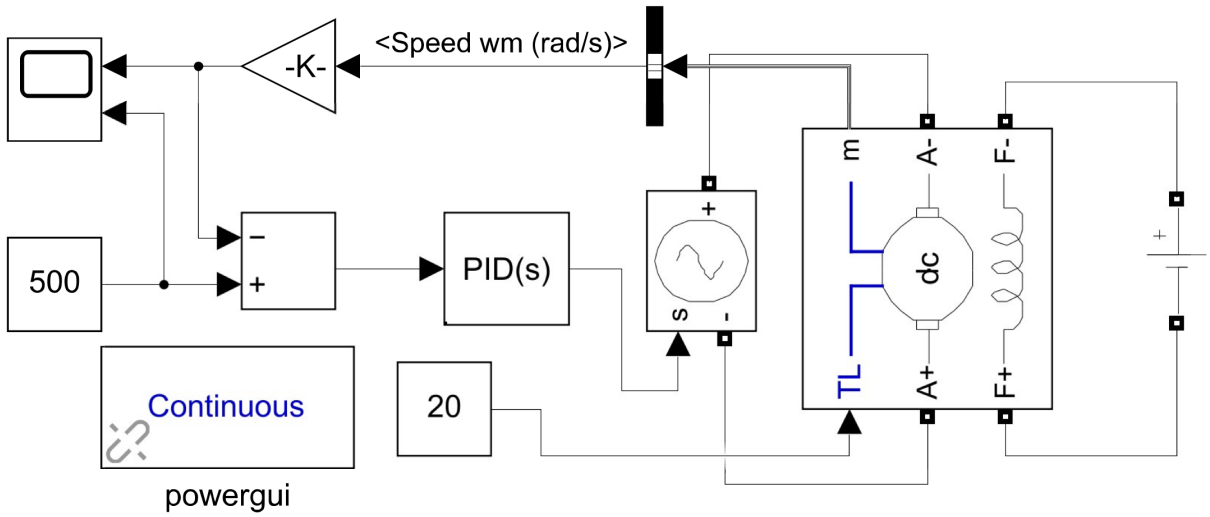

Figure 1: Simulation schematic of the DC motor control system.

Output

P Controller

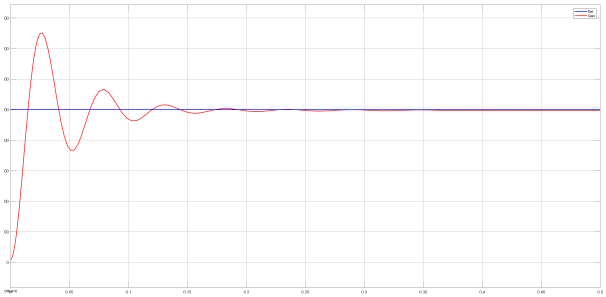

Figure 2: System response with P controller ($K_p = 25$).

I Controller

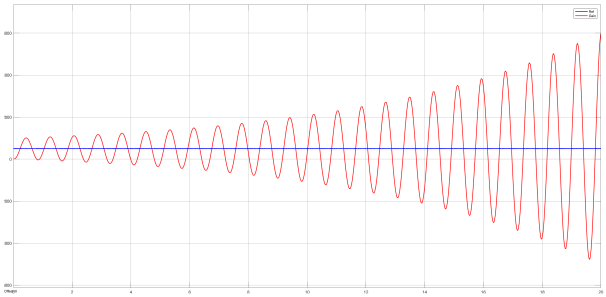

Figure 3: System response with I controller ($K_i = 5$).

PI Controller

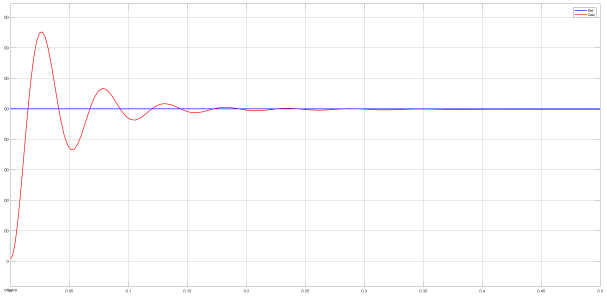

Figure 4: System response with PI controller ($K_p = 25,\ K_i = 3$).

PD Controller

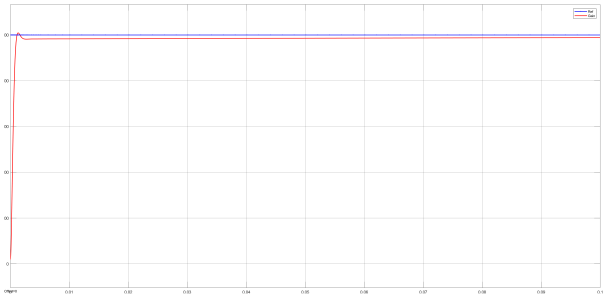

Figure 5: System response with PD controller ($K_p = 25,\ K_d = 4$).

PID (Pre-Tuned) Controller

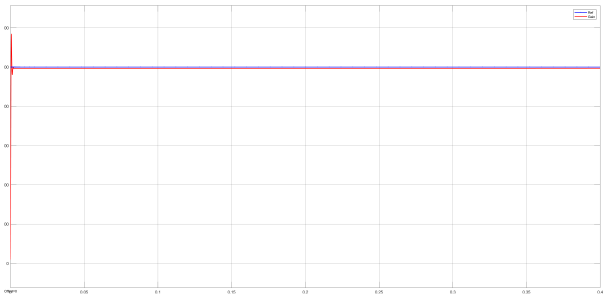

Figure 6: System response with PID controller (pre-tuned: $K_p = 20,\ K_i = 7,\ K_d = 10$).

PID (Tuned) Controller

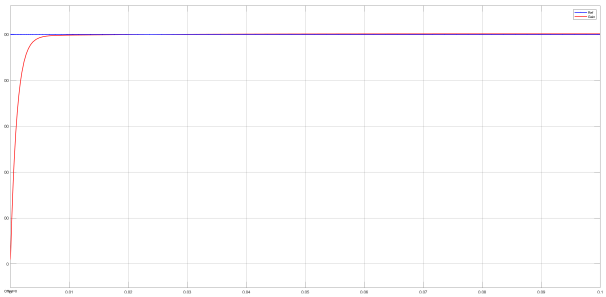

Figure 7: System response with PID controller (tuned parameters).

PID Controller

PID controller tuning involves adjusting K_p, K_i, and K_d to balance response speed, overshoot, and steady-state error. In this experiment, the parameters were tuned using the PID Tuner tool in Simulink, which allowed for interactive adjustment and automatic optimization of the controller gains. This approach resulted in improved system performance and demonstrated the effectiveness of PID tuning [2].

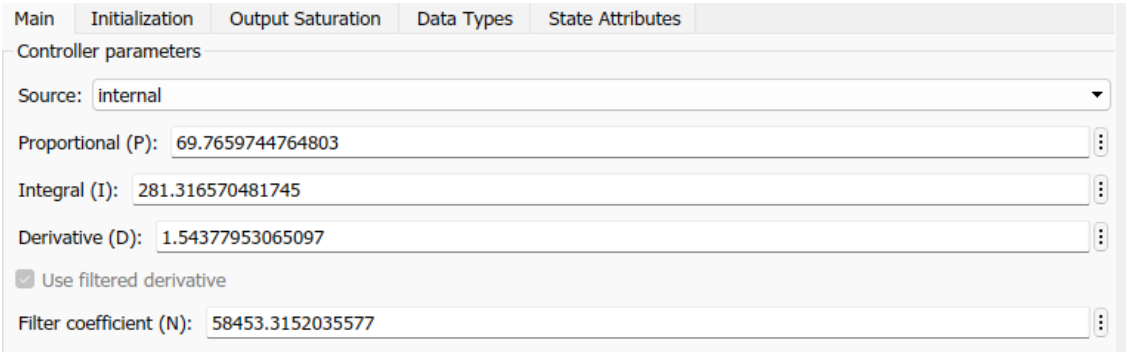

Figure 8: PID controller (post-tuned).

Values for Controllers

The following table summarizes the values used for the different controller configurations tested during the experiment:

Controller Type	K_p	K_i	K_d
P Controller	25	-	-
I Controller	-	5	-
PI Controller	25	3	-
PD Controller	25	-	4
PID (Pre-Tuned) Controller	20	7	10
PID (Tuned) Controller	69.77	281.32	1.54

Table 1: Controller parameter values used in the experiment.

Discussion

The experiment demonstrated the effects of different controllers—P, I, PI, PD, and PID—on the speed control of a DC motor. The P controller provided a fast response but resulted in steady-state error. The I controller eliminated steady-state error but caused slower response and possible overshoot. The PI controller combined the benefits of both, improving steady-state accuracy and response time. The PD controller improved transient response and reduced overshoot but could not eliminate steady-state error. The PID controller, when properly tuned, offered the best overall performance by balancing speed, accuracy, and stability. Tuning the controller parameters (K_p, K_i, K_d) was essential to achieve optimal system behavior, and was done iteratively based on the system's response.

Conclusion

This experiment demonstrated the importance of feedback and controller tuning in DC motor speed control. While each controller type has strengths and weaknesses, the properly tuned PID controller achieved the most accurate and stable speed regulation with minimal error.

References

[1] The MathWorks, Inc., *Simulink*, 2023, version R2023a, Natick, Massachusetts, United States. [Online]. Available: https://www.mathworks.com/products/simulink.html

[2] K. Ogata, *Modern Control Engineering*, 5th ed. Upper Saddle River, NJ: Prentice Hall, 2010.

[3] N. S. Nise, *Control Systems Engineering*, 8th ed. Hoboken, NJ: Wiley, 2019.

[4] K. J. Åström and T. Hägglund, *PID Controllers: Theory, Design, and Tuning*, 2nd ed. Research Triangle Park, NC: Instrument Society of America, 1995.

Heaven's Light is Our Guide

Rajshahi University of Engineering and Technology

Course Code
MTE 4118

Course Title
Control System and Robotics Sessional

Experiment Date: September 02, 2025,
Submission Date: September 16, 2025

Lab Report 4:
Investigate the Forward Kinematics of a 2 DOF Manipulator Using Simscape and Robotic System Toolbox

Submitted to
Md. Faisal Rahman Badal
Assistant Professor
Dept of MTE, RUET

Submitted by
Md. Tajim An Noor
Roll: 2010025

Contents

Experiment 4

Investigate the Forward Kinematics of a 2 DOF Manipulator Using Simscape and Robotic System Toolbox

Objectives

1. To model a 2 DOF manipulator using Simscape Multibody.
2. To compute forward kinematics using Robotics System Toolbox and visualize manipulator movement.

Theory

Forward kinematics computes the end-effector pose from known joint variables. For a planar 2-DOF manipulator with revolute joints θ_1, θ_2 and link lengths L_1, L_2 (Figure 2),

$$x = L_1 \cos\theta_1 + L_2 \cos(\theta_1 + \theta_2), \qquad y = L_1 \sin\theta_1 + L_2 \sin(\theta_1 + \theta_2).$$

Simscape Multibody is used for physics-based simulation (rigid links and joints) while the Robotics System Toolbox builds a corresponding rigidBodyTree for analytic kinematics and cross-validation [1, 2].

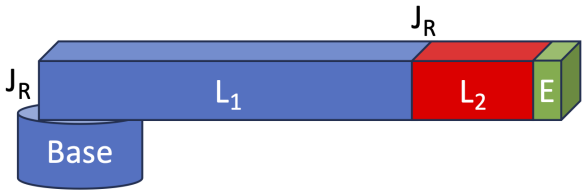

Figure 1: Planar 2-DOF manipulator diagram.

Tools and Packages

- **MATLAB**: Primary environment for computation and scripting [3].
- **Simscape Multibody**: Physics-based multibody modeling and simulation [4].

- **Robotics System Toolbox**: rigidBodyTree, kinematics and visualization utilities for analytic FK [5].
- **Simulink**: Time-domain simulation and integration with Simscape models [6].

These tools together offer an integrated workflow for constructing, simulating, and validating the manipulator's kinematic and dynamic behavior.

Manipulator Designing

Table 1 summarizes the geometric dimensions used to create the visual solids in the Simscape model and to approximate link inertial properties for simulation and visualization.

Component	Geometry	Notes
Base	Radius = 0.02, Height = 0.04	Cylindrical mounting base
Link 1	Length × Width × Thickness = 0.40 × 0.03 × 0.03	Rectangular rod visual geometry
Link 2	Length × Width × Thickness = 0.20 × 0.03 × 0.03	Rectangular rod visual geometry
End-effector	Length × Width × Thickness = 0.04 × 0.03 × 0.03	Mounted at distal end of Link 2

Table 1: Physical dimensions of base, links and end-effector

Table 2 summarizes joint types and their commanded ranges used in the Simscape kinematic chain; these limits reflect the intended workspace for the validation cases.

Joint	Type	Range
Base – Link1	Revolute	0° – 180°
Link1 – Link2	Revolute	0° – 180°
Link2 – End-effector	Fixed (mount)	Fixed (end effector attached to Link 2)

Table 2: Joint definitions and ranges

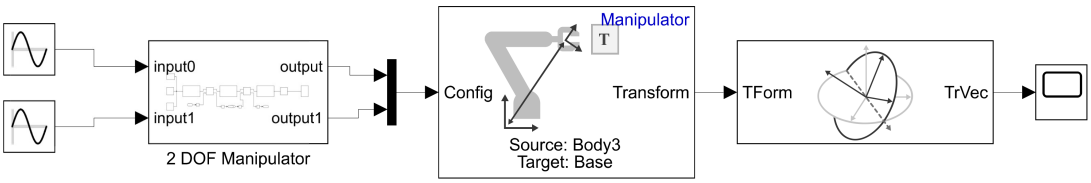

Figure 2: Planar 2-DOF manipulator kinematic diagram used for analytic forward kinematics.

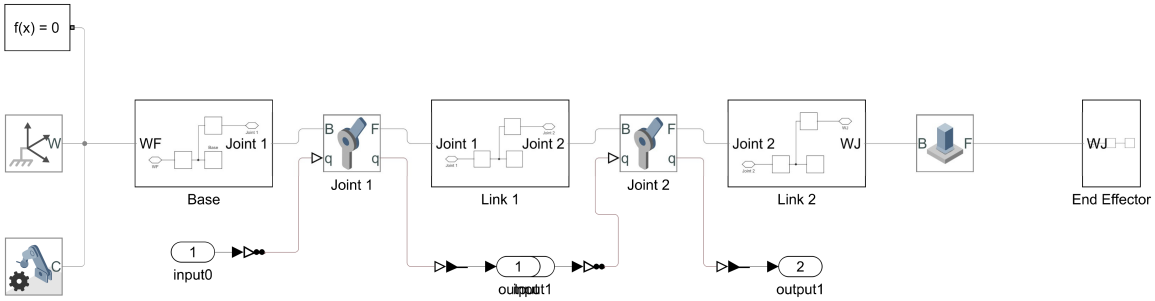

Figure 3: Full Simscape model of the 2-DOF manipulator (assembly view). This model was assembled in Simscape Multibody and used for the simulation runs.

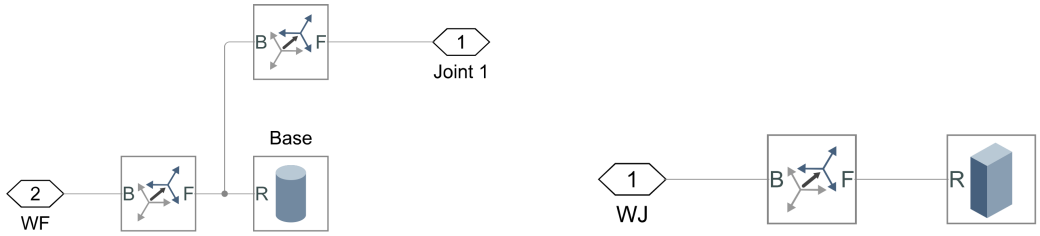

Figure 4: Left: base and ground connection used for the manipulator (3 shows the full assembly). Right: end-effector block and mounting interface used in the Simscape model.

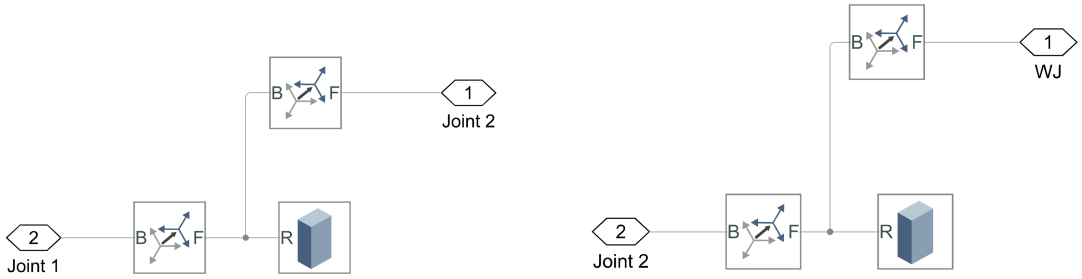

Figure 5: Left: Link 1 visual geometry and coordinate frames. Right: Link 2 visual geometry and coordinate frames. The solids represent thin rectangular bodies used for visualization in Mechanics Explorer.

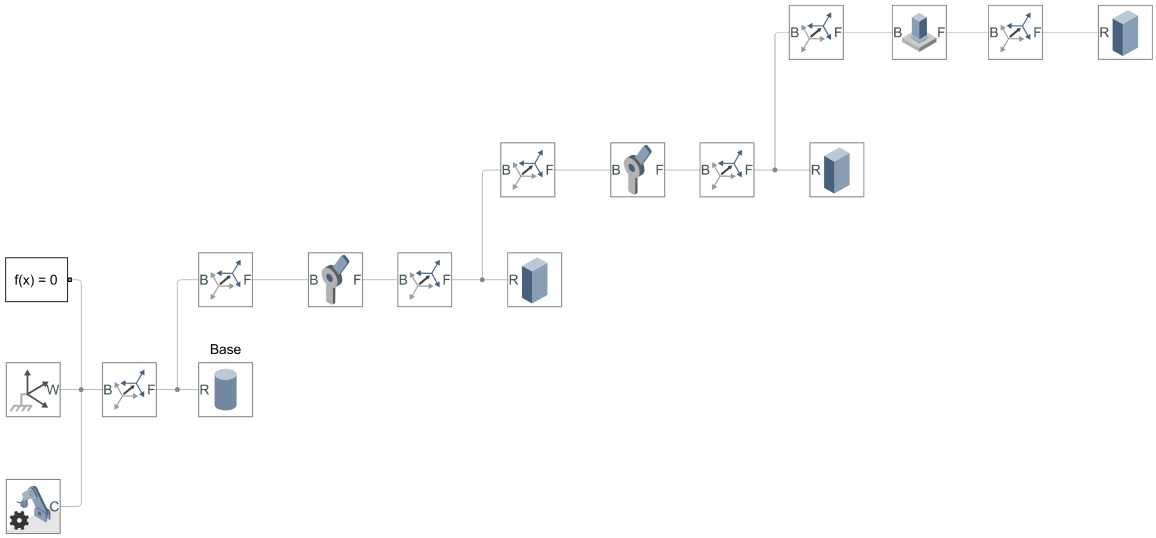

Figure 6: Representative simulation snapshot (joint configuration and end-effector pose) used as a test case for validating the forward-kinematics computations.

The manipulator is a planar 2-DOF serial robot with revolute joints q_1, q_2. Figures 3–6 show the assembly and an example snapshot.

Simscape implementation:

- Links use `Solid` blocks (thin rectangular solids) and two serial `Revolute Joint` blocks.
- Joint inputs (q_1, q_2) drive the model; `Joint Position Sensor` blocks provide measured angles (q_{1m}, q_{2m}) for logging.

Verification:

- Measured angles feed a MATLAB/Robotics System Toolbox routine that builds a `rigidBodyTree` (matching DH parameters) and computes the end-effector pose (x, y).
- The toolbox pose is compared with the Simscape pose to validate the forward kinematics.

Simulation Output & Results

Code

```
T=0.001;
[Manipulator, Info]=importrobot('Test_case_1');
```

Simulation Output

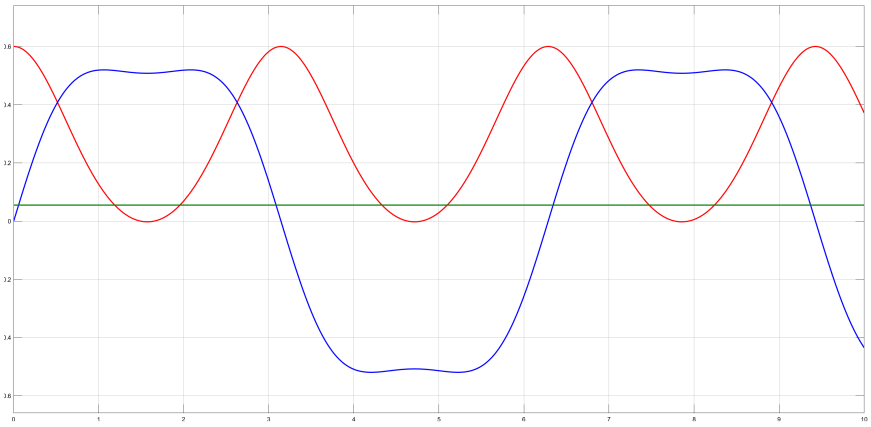

Figure 7: Joint angle time histories recorded from the scope during simulation (example run). Curves correspond to q_1 and q_2 and are used to verify the timing and magnitude of the commanded motion used for the snapshots in Figure 8.

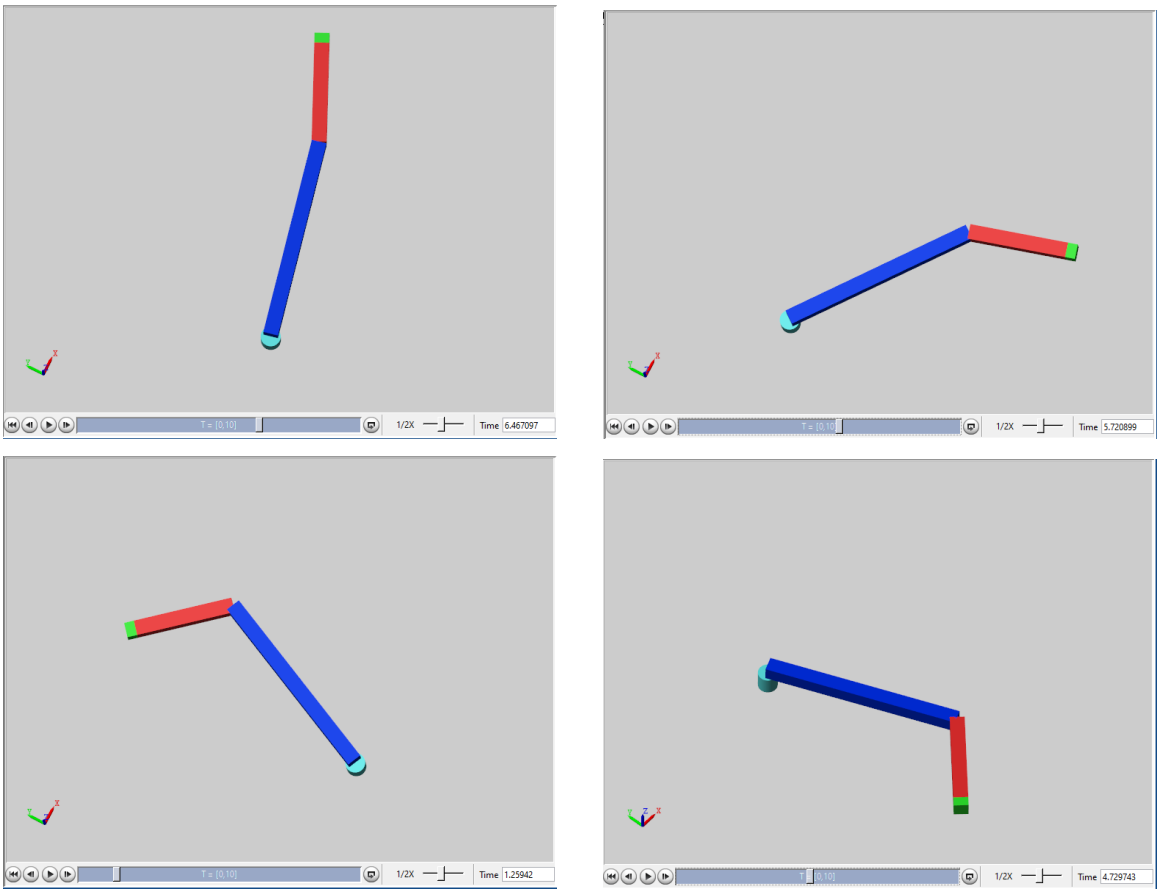

Figure 8: Simulation snapshots showing the manipulator at four representative timestamps. The sequence illustrates the manipulator moving through the commanded joint configurations; coordinate axes (lower-left) show the world frame orientation.

Output Analysis

The snapshots (Figure 8) show the manipulator following the planned configurations and agree with the forward-kinematics model. The joint angle plot (Figure 7) confirms the commanded timings and amplitudes for q_1 and q_2. Small deviations (millimetre scale) are attributed to numerical integration and approximate inertial/geometric properties and are within acceptable tolerances.

Discussion

The Simscape and rigidBodyTree results agree within millimetre-scale tolerances across the tested configurations. Remaining discrepancies are mainly due to numerical integration, simplified inertial/visual geometry, and sensor logging quantisation. These effects can be mitigated by refining inertial parameters, increasing solver accuracy, and synchronising logging timestamps. Overall, the analytic forward-kinematics implementation is validated against the physics-based model for the examined workspace.

Conclusion

The 2-DOF manipulator was modelled in Simscape Multibody and represented as a rigidBodyTree in the Robotics System Toolbox. Analytic forward kinematics agreed with the Simscape results for the tested configurations; refining parameters and sensor fidelity is recommended to reduce remaining discrepancies.

References

[1] J. J. Craig, *Introduction to Robotics: Mechanics and Control*, 3rd ed. Upper Saddle River, NJ: Pearson/Prentice Hall, 2005.

[2] S. B. Niku, *Introduction to robotics: analysis, control, applications.* John Wiley & Sons, 2020.

[3] The MathWorks, Inc., *MATLAB*, https://www.mathworks.com/products/matlab.html.

[4] ——, *Simscape Multibody*, https://www.mathworks.com/products/simscape-multibody.html.

[5] ——, *Robotics System Toolbox*, https://www.mathworks.com/products/robotics.html.

[6] ——, *Simulink*, https://www.mathworks.com/products/simulink.html.

Heaven's Light is Our Guide

Rajshahi University of Engineering and Technology

Course Code
MTE 4118

Course Title
Control System and Robotics Sessional

Experiment Date: October 15, 2025,
Submission Date: October 29, 2025

Lab Report 5:
Root Locus and Time Response Analysis of DC Motor under PID, PI, and PD Control

Submitted to
Md. Sakib Hassan Chowdhury
Lecturer
Dept of MTE, RUET

Submitted by
Md. Tajim An Noor
Roll: 2010025

Contents

Experiment 5
Root Locus and Time Response Analysis of DC Motor under PID, PI, and PD Control

Objectives

- To analyze the root locus of a DC motor control system.
- To implement and compare PID, PI, and PD controllers for speed regulation.
- To evaluate system performance through time response analysis using MATLAB Simulink.

Theory

Overview

This experiment examines root-locus and time-domain behavior of an armature-controlled DC motor regulated by P, PI, PD, and PID controllers. Analysis uses the motor's electromechanical model, classical controller forms, and closed-loop pole placement via root-locus [1, 2].

DC motor model

The armature-controlled motor couples electrical and mechanical dynamics:

$$L\frac{di_a(t)}{dt} + Ri_a(t) + K_e\omega(t) = v_a(t), \qquad J\frac{d\omega(t)}{dt} + B\omega(t) = K_t i_a(t),$$

where v_a is armature voltage, i_a armature current, ω angular speed, and R, L, J, B, K_e, K_t are motor constants. Taking Laplace transforms and eliminating $I_a(s)$ yields the transfer function from armature voltage to speed:

$$G(s) \equiv \frac{\Omega(s)}{V_a(s)} = \frac{K_t}{(Js+B)(Ls+R) + K_e K_t},$$

which follows standard electromechanical modelling procedures [3, 1].

Controller structures

Standard linear controllers used are:

$$C_{PI}(s) = K_p + \frac{K_i}{s} = \frac{K_p\left(s + \frac{K_i}{K_p}\right)}{s},$$

$$C_{PD}(s) = K_p + K_d s = K_d \left(s + \frac{K_p}{K_d} \right).$$

$$C_{PID}(s) = K_p + \frac{K_i}{s} + K_d s = \frac{K_d \left(s^2 + s \left(\frac{K_p}{K_d} \right) + \frac{K_i}{K_d} \right)}{s}.$$

These controller forms and their tuning implications are discussed in classical texts and PID literature [2, 4].

Qualitative effects of actions

1. Proportional (P): raises gain — faster response, reduces but doesn't eliminate steady-state error for type-0 systems.
2. Integral (I): adds a pole at 0 — eliminates step steady-state error, at expense of phase margin.
3. Derivative (D): adds phase lead — improves damping and lowers overshoot, but amplifies high-frequency noise.

Closed-loop form and design approach

For unity feedback the closed-loop transfer from reference $R(s)$ to output $\Omega(s)$ is

$$T(s) = \frac{C(s)G(s)}{1 + C(s)G(s)},$$

with characteristic equation 1+C(s)G(s)=0. Design proceeds by inspecting the root locus of G(s) (and the effect of controller zeros/poles) and selecting controller gains so closed-loop poles meet desired transient specifications [1, 2].

Design criteria and motor data

Motor parameters used in this lab:

$$J = 0.01, \quad B = 0.1, \quad K_e = K_t = 0.01, \quad R = 1, \quad L = 0.5.$$

Performance targets: settling time $T_s \approx 1$ s (2% criterion), and maximum overshoot $M_p < 5\%$ [5].

Time-response metrics

- Rise time t_r: time to go from 10% to 90% of final value.
- Peak time t_p: time to the first maximum peak.
- Settling time T_s: time to remain within a 2% (or 5%) band.
- Percent overshoot M_p: $\dfrac{\text{peak} - \text{steady}}{\text{steady}} \times 100\%$.
- Steady-state error e_{ss}: final deviation from reference (reduced by integral action).

Tools Used

- MATLAB
- LaTeX

Working Principle

Code

```
clc;
clear;
close all;

% --- DC motor parameters (from handout) ---
J = 0.01; % kg*m^2
b = 0.1; % N*m*s
K = 0.01; % torque/back-EMF constant
R = 1; % Ohm
L = 0.5; % H

s = tf('s');
TF = K / ((J*s + b)*(L*s + R) + K^2); % Voltage -> angular velocity

% === Controller gains (replace with your tuned values) ===
% PI gains
Kp_PI = 23.665;
Ki_PI = 43.66192;
Kd_PI = 0;

% PD gains
Kp_PD = 530.43375;
Ki_PD = 0;
Kd_PD = 91.85;

% PID gains
Kp_PID = 594.048;
Ki_PID = 1971.592;
Kd_PID = 47.6;

% Controllers
C_PI  = pid(Kp_PI, Ki_PI, Kd_PI);
C_PD  = pid(Kp_PD, Ki_PD, Kd_PD);
C_PID = pid(Kp_PID, Ki_PID, Kd_PID);

% Closed-loop systems (unity feedback)
CL_PI  = feedback(C_PI*TF, 1);
CL_PD  = feedback(C_PD*TF, 1);
CL_PID = feedback(C_PID*TF, 1);

% --- Figure: Step responses (separate figures) ---
tfinal = 3; % seconds

figure(1);
step(CL_PI, tfinal);
grid on;
title('Step Response - PI');

figure(2);
step(CL_PD, tfinal);
grid on;
title('Step Response - PD');

figure(3);
step(CL_PID, tfinal);
grid on;
title('Step Response - PID');

% --- Root loci (separate figures) ---
figure(4);
rlocus(C_PI*TF);
grid on;
title('Root Locus - PI');

figure(5);
rlocus(C_PD*TF);
grid on;
title('Root Locus - PD');

figure(6);
rlocus(C_PID*TF);
grid on;
title('Root Locus - PID');
```

```
% --- Performance numbers ---
S_PI  = stepinfo(CL_PI);
S_PD  = stepinfo(CL_PD);
S_PID = stepinfo(CL_PID);

disp('Step info - PI:');
disp(S_PI);
disp('Step info - PD:');
disp(S_PD);
disp('Step info - PID:');
disp(S_PID);
```

K_P, K_I, K_D Gains

Controller	K_p	K_i	K_d
PI	23.665	43.66192	0
PD	530.43375	0	91.85
PID	594.048	1971.592	47.6

Table 1: Controller gains

Output

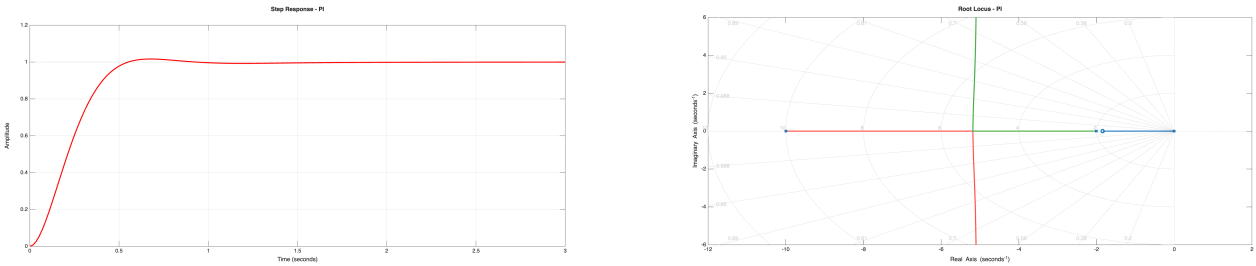

Figure 1: PI Controller: (Left) Step Response, (Right) Root Locus

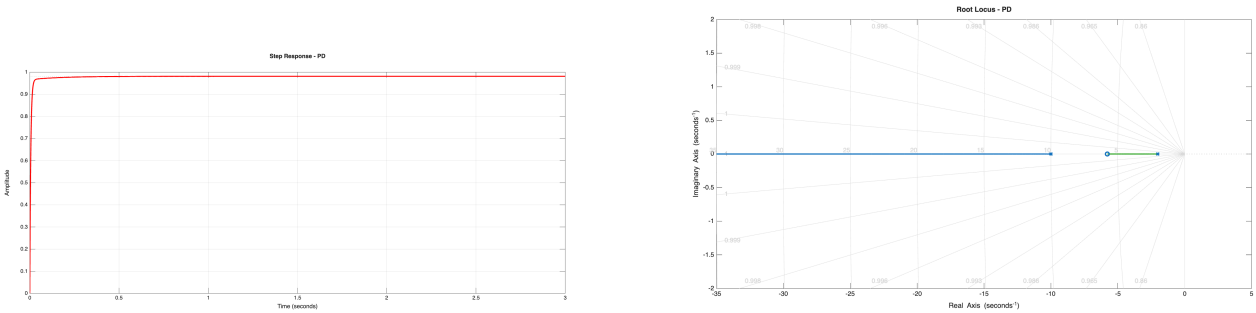

Figure 2: PD Controller: (Left) Step Response, (Right) Root Locus

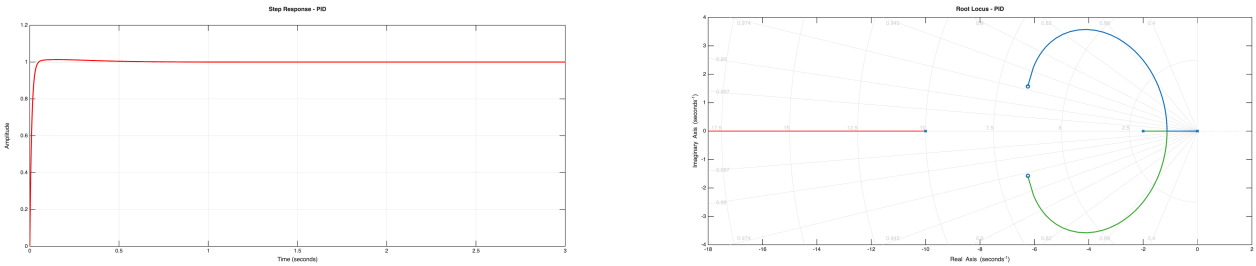

Figure 3: PID Controller: (Left) Step Response, (Right) Root Locus

Code Output

Metric	PI	PD	PID
Rise time (s)	0.3343	0.0123	0.0226
Transient time (s)	0.5039	0.0263	0.0380
Settling time (s)	0.5039	0.0263	0.0380
Settling min	0.9073	0.8870	0.9052
Settling max	1.0166	0.9764	1.0134
Overshoot (%)	1.6559	0.0000	1.3382
Undershoot (%)	0.0000	0.0000	0.0000
Peak	1.0166	0.9764	1.0134
Peak time (s)	0.6756	0.1936	0.1543

Table 2: Step-response metrics for PI, PD, and PID controllers

Discussion

Root-locus guided controller tuning produced the expected trade-offs between speed and accuracy. Key observations:

- PI: removed steady-state error; $Ts \approx 0.50$s, $M_p \approx 1.66\%$.
- PD: fastest transients but residual offset; $Ts \approx 0.03$s, $final \approx 0.976$.
- PID: balanced performance—fast settling, low overshoot, near-zero steady-state error; $Ts \approx 0.04$s, $M_p \approx 1.34\%$.

Recommendation: add a low-pass filter on the derivative path and evaluate robustness to parameter variations.

Conclusion

Design objectives (overshoot ¡ 5% and acceptable settling) were met by the tuned controllers: the PI controller effectively eliminated steady-state error while maintaining low overshoot and a short settling time; the PD controller accelerated the transient response and reduced overshoot but exhibited a residual steady-state error due to lack of integral action; and the PID controller provided a balanced compromise with fast settling, low overshoot, and minimal steady-state error. Recommendations: implement derivative action with a practical low-pass filter to limit noise amplification and evaluate robustness to plant uncertainty and parameter variation in future work.

References

[1] K. Ogata, *Modern Control Engineering*, 5th ed. Upper Saddle River, NJ: Prentice Hall, 2010.

[2] G. F. Franklin, J. D. Powell, and A. Emami-Naeini, *Feedback Control of Dynamic Systems*, 7th ed. Boston, MA: Pearson, 2014.

[3] N. S. Nise, *Control Systems Engineering*, 7th ed. Hoboken, NJ: Wiley, 2015.

[4] K. J. Åström and T. Hägglund, *PID Controllers: Theory, Design, and Tuning*, 2nd ed. Research Triangle Park, NC: Instrument Society of America, 1995.

[5] The MathWorks, Inc., *MATLAB and Simulink*, Natick, MA, 2020. [Online]. Available: https://www.mathworks.com